

DOCUMENTO DELLE MODIFICHE

Coerenza tra il diagramma delle classi UML e l'implementazione

1. Scopo del documento

Il diagramma delle classi UML presentato in fase di analisi definisce, per ciascuna entità del dominio, un insieme di responsabilità (operazioni). In fase di implementazione, alcune di queste responsabilità sono state realizzate integralmente, altre sono state realizzate in forma diversa da un metodo dedicato, e altre ancora non sono state implementate. Questo documento riporta, in modo esplicito e motivato, come ciascuna operazione prevista dall'UML è stata trattata nel codice, distinguendo tra implementazione diretta, implementazione tramite meccanismi equivalenti e mancata implementazione con relativa giustificazione.

2. Operazioni implementate

2.1 Attività (classe base) e sottoclassi

Le operazioni verificare disponibilità posti() e fornire dettagli attività() sono implementate: la disponibilità dei posti è verificata dal controller prima di ogni prenotazione, confrontando il numero di posti richiesti con il valore corrente di posti_disponibili; i dettagli dell'attività sono mostrati nel pannello di dettaglio del Catalogo.

Le operazioni specifiche delle sottoclassi AttivitaCulturale (fornire informazioni sulla tematica()) e AttivitaNaturalistica (indicare la stagione consigliata()) sono state inizialmente trascurate nella prima versione dell'interfaccia grafica, poiché i relativi attributi (tematica, stagioneMigliore) erano presenti nel model ma non ancora esposti a video. Sono state successivamente aggiunte al pannello di dettaglio attività, che ora mostra un'informazione aggiuntiva calcolata in base al tipo concreto dell'oggetto Attività ricevuto, tramite un controllo instanceof sul tipo effettivo (AttivitaCulturale, AttivitaEstrema, AttivitaNaturalistica), unito alla lettura dei rispettivi getter. Questa soluzione sfrutta il **polimorfismo**: lo stesso metodo di visualizzazione si comporta in modo diverso a seconda della sottoclasse concreta ricevuta come parametro di tipo Attività.

2.2 Utente

Le operazioni gestire il proprio profilo(), effettuare prenotazioni(), scrivere recensioni() e creare itinerari() sono tutte implementate. Come discusso in fase di progettazione, ciascuna di esse non corrisponde a un metodo dentro la classe Utente, bensì a un metodo nella DAO dell'oggetto effettivamente prodotto o modificato (rispettivamente UtenteDAO.aggiorna(), PrenotazioneDAO.inserisci(), RecensioneDAO.inserisci(), ItinerarioDAO.inserisci()), orchestrato dal Controller.

2.3 Prenotazione, Recensione, Itinerario, Destinazione

- registrare la prenotazione() e calcolare il prezzo totale() → implementate nel Controller (calcolo del prezzo come prezzo unitario per numero posti) e in PrenotazioneDAO.inserisci()
- aggiornare lo stato della prenotazione() → implementata (metodo di annullamento, che riporta lo stato a ANNULLATA e restituisce i posti disponibili)
- registrare la valutazione utente() → implementata in RecensioneDAO.inserisci()
- gestire tappe del viaggio() → implementata nei metodi aggiungiTappa() / rimuoviTappa() di ItinerarioDAO
- elencare attività disponibili() → implementata tramite AttivitaDAO.trovaPerDestinazione() e i filtri di ricerca nel Catalogo

3. Operazioni non implementate

Le seguenti operazioni, pur previste nel diagramma UML come responsabilità concettuali delle rispettive classi, non sono state implementate come funzionalità eseguibili nel programma. Per ciascuna si riporta la motivazione.

Operazione	Classe	Motivazione
verifica idoneità utente()	AttivitaEstrema	Richiederebbe di confrontare i requisiti dell'attività (livello di rischio, attrezzatura) con caratteristiche dell'utente (es. certificazioni, condizioni fisiche) che non fanno parte del modello Utente. L'operazione resta una responsabilità di dominio corretta, ma la sua implementazione richiederebbe un'estensione del modello dati non prevista nel perimetro di questo progetto.
verificare la propria disponibilità in una data()	GuidaLocale	È una funzionalità lato guida/amministratore, utile per assegnare guide alle attività o gestire sovrapposizioni di calendario. Il progetto implementa un solo attore attivo (l'utente viaggiatore): l'assegnazione delle guide alle attività avviene a monte, in fase di inserimento dati, e non dinamicamente durante l'uso dell'applicazione.
condurre le attività assegnate()	GuidaLocale	Rappresenta un'azione del mondo reale (la guida accompagna fisicamente i partecipanti), non un'operazione software che produce una transazione tracciabile. Il sistema registra l'esito di questa azione tramite il cambio di stato della prenotazione a COMPLETATA, ma non necessita di un metodo che la esegua esplicitamente, così come non è previsto un metodo per un'azione fisica del turista.

4. Considerazioni conclusive

Il criterio applicato in modo uniforme a tutte le scelte descritte in questo documento è il seguente: un'operazione del diagramma UML deve tradursi in un metodo eseguibile quando rappresenta un'azione che il sistema software è in grado di svolgere e verificare autonomamente sui propri dati; quando invece l'operazione presuppone dati non modellati, un attore non previsto dal perimetro applicativo, oppure un'azione puramente fisica del mondo reale, l'operazione resta corretta a livello di analisi del dominio ma non richiede una controparte diretta nel codice. Le uniche due operazioni inizialmente trascurate (tematica e stagione consigliata delle sottoclassi di Attività) sono state individuate e aggiunte all'interfaccia grafica nel corso dello sviluppo, ripristinando la piena coerenza con il diagramma UML su questo punto.

